

RISC-V S 模式 特权架构详解

ISA 定义 · 硬件实现 · 操作系统编程

基于 RISC-V Privileged ISA Spec V20211203

本文档完整覆盖 RISC-V S 模式（Supervisor Mode）的指令集架构定义、
CPU 硬件实现方法，以及操作系统软件编程实践。
三部分形成「规格—实现—使用」的完整闭环。

目录

I	S 模式指令集架构定义	5
1	S 模式总览	5
1.1	S 模式的设计定位	5
1.2	特权级与 S 模式的关系	5
1.3	S 模式的最小实现要求	6
2	S 模式 CSR 详解	6
2.1	sstatus — 监管者状态寄存器（地址 0x100）	6
2.2	sie / sip — 中断使能与挂起（地址 0x104 / 0x144）	7
2.3	stvec — 陷阱向量基址寄存器（地址 0x105）	7
2.4	sscratch — 暂存寄存器（地址 0x140）	7
2.5	sepc / scause / stval — 陷阱上下文三件套	7
2.6	satp — 监管者地址转换与保护（地址 0x180）	8
2.7	stimecmp — S 模式定时器比较器（地址 0x14D）	8
2.8	senvcfg — 环境配置寄存器（地址 0x10A）	9
2.9	scontext — 上下文寄存器（地址 0x5A8）	9
3	S 模式特权指令	9
3.1	SRET — 监管者异常返回	9
3.2	SFENCE.VMA — 地址翻译同步指令	9
3.3	ECALL 在 S 模式的语义	10
4	陷阱委托机制	10
4.1	medeleg / mideleg 的作用	10
4.2	委托的硬件行为	10
4.3	典型委托配置	10
5	Sv32 虚拟内存系统	11
5.1	Sv32 的地址格式	11
5.2	Sv32 页表项（PTE）格式	11
5.3	Sv32 页表遍历算法	11
5.4	权限检查的细节	12
5.5	A/D 位的两种处理方式	13
5.6	多级页表的物理内存布局	13
II	CPU 硬件实现	13

6	CSR 文件硬件设计	13
6.1	S 模式 CSR 的物理实现	13
6.2	sstatus 与 mstatus 的共享视图	13
6.3	CSR 读写逻辑的 RTL 实现	14
7	陷阱处理硬件	16
7.1	陷阱检测逻辑	16
7.2	陷阱状态机	16
7.3	SRET 执行硬件	17
8	页表遍历器 (PTW) 设计	17
8.1	PTW 的整体架构	17
8.2	PTW 与流水线的交互	18
8.3	缺页异常的生成	18
9	TLB 设计	19
9.1	TLB 的基本结构	19
9.2	TLB 查询与命中	19
9.3	SFENCE.VMA 的 TLB 刷新	19
9.4	ASID 支持	20
10	地址翻译与流水线集成	20
10.1	取指阶段的翻译	20
10.2	访存阶段的翻译	20
10.3	翻译与 PMP 的顺序	20
10.4	性能考量与优化	21
III	操作系统软件编程	21
11	从 M 模式到 S 模式的启动	21
11.1	启动的整体流程	21
11.2	OpenSBI 的角色	21
11.3	M 模式到 S 模式的跳转代码	22
11.4	S 模式入口的初始化	22
12	OS 陷阱处理框架	23
12.1	stvec 的设置与陷阱入口	23
12.2	寄存器保存与恢复	23
12.3	scause 分发逻辑	24
12.4	系统调用处理	25
12.5	中断处理	26

13 OS 页表管理	27
13.1 页表的分配与初始化	27
13.2 PTE 的填充	28
13.3 虚拟地址映射	28
13.4 satp 切换与 SFENCE.VMA	29
13.5 缺页处理	29
14 完整代码示例	30
14.1 最小陷阱处理程序	30
14.2 页表初始化示例	31
14.3 系统调用处理示例	32
14.4 端到端流程：一次系统调用的完整路径	32

Part I

S 模式指令集架构定义

本部分从 ISA 规范的角度，完整定义 RISC-V S 模式（Supervisor Mode）的指令集架构。我们将逐一讲解 S 模式的每一个 CSR、每一条特权指令、陷阱委托机制，以及 Sv32 虚拟内存系统的全部细节。这些定义是硬件实现和软件编程的共同契约——硬件必须精确地按照这些定义来响应，软件必须精确地按照这些定义来编程。理解了本部分，你就拥有了 S 模式的规格说明书，后续的硬件实现和 OS 编程都是对这份说明书的落地。

1 S 模式总览

1.1 S 模式的设计定位

RISC-V 的 S 模式（Supervisor Mode，监管者模式）是为传统操作系统设计的特权级。它的设计哲学是「够用就好」——提供操作系统运行所需的最小特权集，同时把对硬件的完全控制权留给 M 模式。这种分层设计意味着 S 模式不能直接操作机器级寄存器（如 `mstatus`、`mie`、`mtvec`），不能直接配置 PMP，也不能直接处理机器定时器中断。S 模式能做的是：管理自己的中断与异常、管理虚拟内存（Sv32/39/48）、执行 SRET 返回 U 模式、通过 ECALL 向 M 模式请求服务（SBI 调用）。

这种设计的实际意义在于：M 模式运行一个固件（如 OpenSBI），充当硬件抽象层；S 模式运行操作系统内核（如 Linux），通过 SBI 调用间接使用硬件资源。当操作系统需要设置定时器、读写串口、或处理 IPI 时，它不能直接访问 CLINT/UART 寄存器（因为 PMP 可能阻止 S 模式访问），而是通过 ECALL 请求 M 模式固件代为执行。这种间接性虽然增加了一些开销，但带来了清晰的隔离：固件可以独立更新，操作系统不需要关心硬件细节。

对于个人开发者设计 RV32IM 处理器，S 模式的价值在于：它是运行「真正的操作系统」（而不是裸机监控程序）的最低门槛。一旦你实现了 S 模式 + Sv32 分页 + 陷阱委托，你的处理器就能跑 Linux——这是验证处理器正确性的终极测试。

1.2 特权级与 S 模式的关系

RISC-V 定义了四个特权级，编码为 2 位：

编码	名称说明
11	Machine (M) 最高权限，复位后进入，可访问所有 CSR
01	Supervisor (S) 操作系统运行级，管理虚拟内存与 S 级中断
00	User (U) 用户程序运行级，权限最低
10	Hypervisor (HS) 虚拟化扩展，本文档不涉及

S 模式与 M 模式的关键区别在于「可见性」：S 模式只能访问地址高 2 位为 01 的 CSR（如 `sstatus=0x100`），不能访问高 2 位为 11 的 CSR（如 `mstatus=0x300`）。如果 S 模式代码试图读写 M 级 CSR，硬件会触发非法指令异常。这种隔离是硬件强制的——CSR 地址的高 2 位天然划分了访问权限。

S 模式与 U 模式的关键区别在于「特权指令」: SRET、SFENCE.VMA、ECALL 等指令在 S 模式可以执行,在 U 模式执行会触发非法指令异常(ECALL 除外,它在 U 模式触发 `environment-call-from-U-mode` 异常)。此外, S 模式可以访问 satp CSR 来控制虚拟内存, U 模式不能。

1.3 S 模式的最小实现要求

要实现一个可用的 S 模式, 硬件至少需要:

- **CSR 子集**: sstatus、sie、sip、stvec、sscratch、sepc、scause、stval、satp (共 9 个)。
- **特权指令**: SRET、SFENCE.VMA (Sv32 启用后)。
- **陷阱委托**: medeleg 和 mideleg 寄存器 (M 模式配置, 将特定陷阱委托给 S 模式)。
- **虚拟内存**: satp 的 Sv32 模式、页表遍历器 (PTW)、TLB (可选但强烈推荐)。
- **特权级切换**: 硬件在陷阱和 SRET 时自动切换 Current Privilege Level (CPL)。

如果只实现 M+U 两级 (不实现 S), 则不需要上述任何内容, 但也不能运行标准 OS。对于「一生一芯」计划, 实现 S 模式是跑通 Linux 的前提。

2 S 模式 CSR 详解

S 模式的 CSR 地址范围是 0x100–0x1FF (高 2 位 = 01)。以下逐一讲解初学者必须实现的核心 CSR。

2.1 sstatus — 监管者状态寄存器 (地址 0x100)

sstatus 是 mstatus 的「S 模式视图」——它不是独立的寄存器, 而是 mstatus 中与 S 模式相关位的子集。读写 sstatus 实际上是在读写 mstatus 的对应位。

位	名称含义	对应 mstatus 位
1	SIE	S 模式全局中断使能。1= 允许 S 级中断
5	SPIE	陷阱前的 SIE 旧值。SRET 时恢复
8	SPP	陷阱前的特权级。0= 来自 U, 1= 来自 S
17	SUM	S 模式是否允许访问 U 模式页。0= 禁止
18	MXR	可执行页是否可读。0= 按 R 位检查
19	TSR	陷阱 SRET。1=S 模式 SRET 触发异常

SIE/SPIE/SPP 的陷阱机制: 当 hart 因陷阱进入 S 模式时, 硬件自动执行: $SPIE \leftarrow SIE$; $SIE \leftarrow 0$; $SPP \leftarrow CPL$ (0 或 1)。这保证了 S 模式陷阱处理过程中不会被同级中断打断。当执行 SRET 时, 硬件反向恢复: $CPL \leftarrow SPP$; $SIE \leftarrow SPIE$; $SPIE \leftarrow 1$ 。

SUM 位: 默认情况下, S 模式 (内核) 不能访问 U 模式页表项标记的页面。这防止了内核意外读写用户数据。当内核需要访问用户内存 (如 `copy_to_user`) 时, 临时设置 SUM=1, 访问完毕后清除。

MXR 位: 正常情况下, 一个标记为 R=0、X=1 的页面 (只执行页) 不可读。设置 MXR=1 后, X=1 的页面也被视为可读。这用于某些需要读取可执行页内容的场景 (如 ELF 加载器读取 `.text` 段)。

2.2 sie / sip — 中断使能与挂起（地址 0x104 / 0x144）

sie (Supervisor Interrupt Enable) 和 sip (Supervisor Interrupt Pending) 是 S 模式的中断使能和挂起寄存器，结构与 mie/mip 对应。

位	名称	sie 含义	sip 含义
1	SSIE/SSIP	S 级软件中断使能	S 级软件中断挂起（可读写）
5	STIE/STIP	S 级定时器中断使能	S 级定时器中断挂起（只读）
9	SEIE/SEIP	S 级外部中断使能	S 级外部中断挂起（只读）

SSIP（软件中断）在 sip 中是可读写的——软件可以直接写 1 来触发 S 级软件中断，这常用于核间中断（IPI）。**STIP**（定时器中断）和 **SEIP**（外部中断）在 sip 中是只读的，由外部硬件（CLINT/PLIC）设置和清除。

中断触发条件：S 级中断 i 实际触发需要同时满足：(a) `sstatus.SIE=1`（全局使能）；(b) `sie[i]=1`（个体使能）；(c) `sip[i]=1`（挂起）；(d) 当前 $CPL \leq S$ 模式（即 U 模式或 S 模式且 $SIE=1$ ）；(e) `mideleg[i]=1`（已委托给 S 模式）。

2.3 stvec — 陷阱向量基址寄存器（地址 0x105）

stvec 告诉硬件「S 模式陷阱发生时跳到哪里」。它由两部分组成：

- **BASE** (bit 31:2)：30 位基址，必须 4 字节对齐。
- **MODE** (bit 1:0)：模式选择。

MODE	名称	入口地址计算
00	Direct	$PC \leftarrow \text{BASE}$ （所有陷阱跳同一地址）
01	Vectored	中断： $PC \leftarrow \text{BASE} + 4 \times \text{cause}$ ；异常： $PC \leftarrow \text{BASE}$

Direct 模式最简单，所有陷阱跳到 BASE，由软件读 `scause` 分发。Vectored 模式省去软件分发开销，但要求 BASE 4 字节对齐且每个向量槽至少 4 字节。初学者推荐 Direct 模式。

2.4 sscratch — 暂存寄存器（地址 0x140）

sscratch 是一个纯通用的 32 位读写寄存器，硬件不对其做任何特殊处理。它的典型用途是保存「S 模式栈指针」——当 U 模式程序触发陷阱进入 S 模式时，陷阱入口的第一条指令是 `crrw sp, sscratch, sp`，原子地交换 `sp` 和 `sscratch`，于是 `sp` 指向内核栈，`sscratch` 保存了被中断的用户 `sp`。退出时再交换一次恢复。

2.5 sepc / scause / stval — 陷阱上下文三件套

这三个寄存器在陷阱进入 S 模式时由硬件自动写入：

sepc 的关键细节：对于异常，sepc 指向触发异常的那条指令本身（SRET 后会重新执行它，适用于缺页重试）。对于中断，sepc 指向中断返回后应执行的下一条指令（因为中断不是错误，不应重新执行被打断的指令）。如果异常发生在压缩指令（C 扩展）上，sepc 的最低位为 1；非压缩指令则为 0。

寄存器	地址	硬件写入内容
-----	----	--------

sepc	0x141	触发陷阱的指令地址（异常）或下一条指令地址（中断）
scause	0x142	陷阱原因码（最高位 1= 中断，0= 异常；低 31 位为编号）
stval	0x143	附加信息：异常地址、非法指令编码、缺页地址等

scause 的常见值：

码	类型	说明
---	----	----

0	异常	指令地址对齐错误
2	异常	非法指令
3	异常	断点（EBREAK）
5	异常	Load 访问错误
7	异常	Store/AMO 访问错误
8	异常	U 模式 ECALL（系统调用）
9	异常	S 模式 ECALL
12	异常	缺页错误（指令获取）
13	异常	缺页错误（Load）
15	异常	缺页错误（Store/AMO）
1	中断	S 级软件中断（SSI）
5	中断	S 级定时器中断（STI）
9	中断	S 级外部中断（SEI）

2.6 satp — 监管者地址转换与保护（地址 0x180）

satp 是 Sv32 虚拟内存的控制开关。它的格式（RV32）如下：

字段	位含义
----	-----

MODE	31	0=Bare（不翻译），1=Sv32（启用两级页表）
ASID	30:22	9 位地址空间标识符，用于 TLB 区分进程
PPN	21:0	22 位根页表物理页号（基址 $\gg 12$ ）

写 satp 会切换页表——这是进程切换的核心操作。OS 在切换进程时，将新进程的根页表物理地址写入 satp.PPN，设置 MODE=1（Sv32），然后执行 SFENCE.VMA 刷新 TLB。

satp 在 S 模式可读写，在 U 模式不可访问。M 模式访问 satp 不会触发地址翻译（M 模式始终使用物理地址）。

2.7 stimecmp — S 模式定时器比较器（地址 0x14D）

stimecmp 是 RISC-V Privileged Spec 1.12 新增的 CSR，用于 S 级定时器中断。当全局计时器 mtime $\geq$ stimecmp 时，硬件置 sip.STIP=1，触发 S 级定时器中断。OS 通过写 stimecmp 设置下一次定时器中断的时间点。

在旧版规范中，S 级定时器中断需要通过 SBI 调用请求 M 模式设置 `mtimecmp`。`stimecmp` 的引入让 S 模式可以直接管理定时器，减少 SBI 调用开销。对初学者，如果 M 模式固件不支持 `stimecmp`，可以继续使用 SBI 方式。

2.8 senvcfg — 环境配置寄存器（地址 0x10A）

`senvcfg` 控制 U 模式的某些行为，如 FENCE 指令对 I/O 的语义（FIOM 位）、缓存零操作（CBZE）等。初学者可以不实现此寄存器（返回 0）。

2.9 scontext — 上下文寄存器（地址 0x5A8）

`scontext` 是一个供调试器使用的可选寄存器，用于关联硬件上下文与软件上下文。初学者不需要实现。

3 S 模式特权指令

3.1 SRET — 监管者异常返回

SRET 是 S 模式的陷阱返回指令，执行以下硬件操作：

1. $CPL \leftarrow sstatus.SPP$ （恢复到陷阱前的特权级）
2. $PC \leftarrow sepc$ （返回到陷阱点）
3. $sstatus.SIE \leftarrow sstatus.SPIE$ （恢复中断使能）
4. $sstatus.SPIE \leftarrow 1$
5. $sstatus.SPP \leftarrow 0$ （U 模式）

如果 `sstatus.TSR=1`，S 模式执行 SRET 会触发非法指令异常。这允许 M 模式禁止 S 模式自行返回 U 模式（强制通过 M 模式中转）。初学者通常设 `TSR=0`。

SRET 是特权指令——在 U 模式执行会触发非法指令异常。编码：`funct12=0x102000`，`opcode=1110011`。

3.2 SFENCE.VMA — 地址翻译同步指令

SFENCE.VMA 指令用于同步页表修改与后续地址翻译。它的语义是：确保此指令之前的所有页表写入，对后续的地址翻译可见。

Listing 1: SFENCE.VMA 指令格式

```
1 sfence.vma rs1, rs2 # rs1=虚拟地址, rs2=ASID
2 sfence.vma x0, x0 # 刷新所有 ASID 的所有地址
3 sfence.vma ra, x0 # 刷新所有 ASID 中 va=ra 的条目
4 sfence.vma x0, rb # 刷新 ASID=rb 的所有条目
```

OS 在以下场景必须执行 SFENCE.VMA：

- 修改页表项后（PTE 内容变化）
- 切换进程（写 `satp` 后，如果不使用 ASID）
- 释放页表页（避免 TLB 指向已释放的页表）
- 修改 PMP 配置后（如果启用了虚拟内存）

在 U 模式执行 SFENCE.VMA 会触发非法指令异常。如果 sstatus.TSR=1, S 模式执行也会触发异常（但通常不设此位）。

3.3 ECALL 在 S 模式的语义

ECALL 不是 S 模式专用指令，但它在 S 模式有特殊语义。当 S 模式执行 ECALL 时，硬件触发 environment-call-from-S-mode 异常（cause=9）。这个异常通常被委托给 M 模式（medeleg[9]=0），由 M 模式固件处理——这就是 SBI 调用的底层机制。

当 U 模式执行 ECALL 时，触发 environment-call-from-U-mode 异常（cause=8），通常委托给 S 模式（medeleg[8]=1），由 OS 的系统调用处理程序处理。

4 陷阱委托机制

4.1 medeleg / mideleg 的作用

默认情况下，所有陷阱都进入 M 模式。但 RISC-V 允许 M 模式将特定陷阱「委托」给 S 模式处理。委托通过两个 M 级 CSR 控制：

- medeleg(Machine Exception Delegation, 地址 0x302): 每一位对应一个异常 cause。medeleg[i]=1 表示 cause=i 的异常直接进入 S 模式。
- mideleg(Machine Interrupt Delegation, 地址 0x303): 每一位对应一个中断 cause。mideleg[i]=1 表示 cause=i 的中断直接进入 S 模式。

4.2 委托的硬件行为

当陷阱发生时，硬件按以下逻辑决定进入哪个模式：

1. 读取 mcause 获取陷阱 cause 编号 c 。
2. 如果是异常：检查 medeleg[c]。如果为 1，进入 S 模式（操作 sstatus/sepc/scause/stval/stvec）；否则进入 M 模式。
3. 如果是中断：检查 mideleg[c]。如果为 1，进入 S 模式；否则进入 M 模式。

4.3 典型委托配置

OpenSBI 在启动时通常设置以下委托：

寄存器	位说明	
medeleg	bit 8	U 模式 ECALL（系统调用）委托给 S
medeleg	bit 9	S 模式 ECALL 不委托（留给 M 处理 SBI）
medeleg	bit 12–15	缺页错误委托给 S
mideleg	bit 1	S 级软件中断（SSI）委托给 S
mideleg	bit 5	S 级定时器中断（STI）委托给 S
mideleg	bit 9	S 级外部中断（SEI）委托给 S

注意：medeleg[11]（M 模式 ECALL）必须为 0——M 模式 ECALL 不能委托给 S 模式，否则会形成无限委托循环。

5 Sv32 虚拟内存系统

5.1 Sv32 的地址格式

Sv32 支持 32 位虚拟地址空间，4 KiB 页大小。虚拟地址分为三段：

位	名称用途	宽度	
31:22	VPN[1]	一级页表索引	10 位
21:12	VPN[0]	二级页表索引	10 位
11:0	offset	页内偏移	12 位

物理地址（Sv32 标准为 34 位，初学者可简化为 32 位）：

位	名称用途	宽度	
33:22	PPN[1]	物理页号高位	12 位
21:12	PPN[0]	物理页号低位	10 位
11:0	offset	页内偏移（从 VA 复制）	12 位

5.2 Sv32 页表项（PTE）格式

每个 PTE 为 32 位，格式如下：

位	名称含义	
0	V	有效位。V=0 表示此 PTE 无效
1	R	可读
2	W	可写
3	X	可执行
4	U	用户模式可访问（U=0 表示 S 模式专用）
5	G	全局映射（所有 ASID 共享，不随进程切换失效）
6	A	已访问位（硬件或软件设置）
7	D	脏位（被写入过，硬件或软件设置）
9:8	RSW	保留给软件使用
31:10	PPN	物理页号（22 位）

叶子节点判断：R、W、X 三位中至少一个为 1 的 PTE 是「叶子节点」（直接映射物理页）。R=W=X=0 的 PTE 是「非叶子节点」（指向下一级页表）。

5.3 Sv32 页表遍历算法

当 `satp.MODE=1`（Sv32）且 CPL 为 S 或 U 时，所有内存访问的虚拟地址都要经过翻译。算法如下：

Listing 2: Sv32 页表遍历算法（伪代码）

```

1 // satp.PPN 是根页表的物理页号
2 pte_addr = satp.PPN * 4096 + VPN[1] * 4 // 一级页表项地址
3 pte = Memory[pte_addr] // 读取一级 PTE
4
5 if (pte.V == 0) goto page_fault // 无效 PTE
6
7 if (pte.R == 0 && pte.W == 0 && pte.X == 0) {
8     // 非叶子节点, 指向二级页表
9     pte_addr = pte.PPN * 4096 + VPN[0] * 4 // 二级页表项地址
10    pte = Memory[pte_addr] // 读取二级 PTE
11    if (pte.V == 0) goto page_fault // 无效 PTE
12 }
13
14 // 此时 pte 是叶子节点
15 // 检查权限
16 if (access_type == READ && pte.R == 0) goto page_fault
17 if (access_type == WRITE && pte.W == 0) goto page_fault
18 if (access_type == FETCH && pte.X == 0) goto page_fault
19
20 // 检查 U 位 (用户/内核隔离)
21 if (CPL == U && pte.U == 0) goto page_fault // U 模式访问 S 页
22 if (CPL == S && pte.U == 1 && sstatus.SUM == 0)
23     goto page_fault // S 模式访问 U 页 (SUM=0)
24
25 // 检查 A/D 位
26 if (pte.A == 0 || (access_type == WRITE && pte.D == 0))
27     goto page_fault // 触发缺页, 由 OS 设置 A/D
28
29 // 翻译成功
30 phys_addr = pte.PPN * 4096 + offset

```

5.4 权限检查的细节

权限检查是 Sv32 安全性的核心, 分三层:

第一层: R/W/X 权限。PTE 的 R、W、X 位定义了该页的访问权限。如果操作类型不匹配 (如写一个 W=0 的页), 触发缺页异常。注意: R=0、W=1 的组合在 Sv32 中是非法的 (写权限隐含读权限), 硬件应将其视为无效 PTE。

第二层: U 位 (用户/内核隔离)。U=1 的页面是用户页面, U 模式可以访问; U=0 的页面是内核页面, 只有 S 模式可以访问。S 模式默认不能访问 U=1 的页面 (除非 sstatus.SUM=1), 这防止了内核意外读写用户数据。

第三层: A/D 位。A 位表示该页已被访问, D 位表示该页已被写入。如果 A=0 (或写操作时 D=0), 触发缺页异常, 由 OS 设置 A/D 位后重试。

5.5 A/D 位的两种处理方式

方式一：硬件自动设置。硬件在遍历时发现 $A=0$ （或写时 $D=0$ ），自动置位并写回 PTE。对软件透明，但需要硬件实现「读 PTE → 修改 → 原子写回」逻辑。

方式二：软件设置（缺页触发）。硬件发现 $A=0$ （或写时 $D=0$ ）时触发缺页异常。OS 缺页处理程序设置 A/D 位，写回 PTE，执行 SFENCE.VMA，返回后重新执行访问指令。硬件简单，但首次访问有额外开销。

初学者推荐方式二——硬件只需在 $A=0$ 或（写且 $D=0$ ）时触发缺页，不需要 PTE 写回逻辑。

5.6 多级页表的物理内存布局

- **根页表：**1 个，4 KiB，1024 个 PTE。satp.PPN 指向它。
- **二级页表：**最多 1024 个（按需分配），每个 4 KiB，1024 个 PTE。
- **PTE 大小：**4 字节。
- **页表对齐：**4 KiB 对齐。

一个进程的完整页表结构最多 4 MiB + 4 KiB，但实际使用是稀疏的——二级页表按需分配。OS 在创建进程时分配一个根页表（4 KiB），初始化所有 PTE 为 0。当进程访问一个虚拟地址且对应的二级页表不存在时，OS 在缺页处理中分配新的 4 KiB 二级页表，填充根 PTE 指向它。这种「按需分配」是虚拟内存的核心优势。

Part II

CPU 硬件实现

本部分从硬件设计者的视角，讲解如何用 RTL（Verilog/Chisel）实现第一部分定义的 S 模式 ISA。我们将覆盖 CSR 文件设计、陷阱处理硬件、页表遍历器（PTW）、TLB 设计，以及地址翻译与流水线的集成。每一节都会给出关键的硬件结构图和设计要点，帮助你把 ISA 规范转化为可综合的硬件。

6 CSR 文件硬件设计

6.1 S 模式 CSR 的物理实现

CSR 文件是处理器中一组专用寄存器的集合，每个 CSR 有唯一的 12 位地址。在硬件实现中，CSR 文件通常是一个多端口的寄存器堆——至少有一个读端口（CSRREAD/CSRRC 读操作）和一个写端口（CSRWRITE/CSRWS/CSRRC 写操作），加上若干硬件读写端口（陷阱时硬件写入 sepc/scause/stval/sstatus，SRET 时硬件读写 sstatus/sepc）。

6.2 sstatus 与 mstatus 的共享视图

sstatus 不是独立寄存器——它是 mstatus 的部分位的「别名」。在硬件中，只有一个 mstatus 寄存器，读写 sstatus 实际上是在读写 mstatus 的特定位。

端口	方向	宽度	用途
csr_addr	输入	12	CSR 地址（来自指令 imm[11:0]）
csr_rdata	输出	32	读出的 CSR 值
csr_wdata	输入	32	要写入的值
csr_we	输入	1	写使能
csr_op	输入	2	操作类型：00= 读不改, 01=CSRRW, 10=CSRRS, 11=CSRRC
trap_enter	输入	1	陷阱进入 S 模式（硬件写 sepc 等）
trap_cause	输入	32	陷阱原因（写入 scause）
trap_epc	输入	32	陷阱 PC（写入 sepc）
trap_tval	输入	32	陷阱附加信息（写入 stval）
sret_exec	输入	1	SRET 执行（硬件读 sepc/sstatus）
priv_level	输出	2	当前特权级（来自 sstatus.SPP 或 M 级等价物）

具体映射关系：

sstatus 位	mstatus 位说明
SIE (bit 1)	MIE (bit 3) S 模式全局中断使能
SPIE (bit 5)	MPPIE (bit 7) 陷阱前 SIE 旧值
SPP (bit 8)	MPP (bit 12:11) 陷阱前特权级（仅最低位）

在 RTL 中，当 csr_addr = 0x100（sstatus）时：

- 读：csr_rdata = {mstatus[8], mstatus[5], ..., mstatus[1]}(将 mstatus 的对应位映射到 sstatus 的位置，其余位补 0)。
- 写：将写入数据的对应位写回 mstatus 的对应位，不影响 mstatus 的其他位。

6.3 CSR 读写逻辑的 RTL 实现

Listing 3: CSR 读写逻辑（Verilog 伪代码）

```

1 module csr_file (
2     input wire      clk,
3     input wire [11:0] csr_addr,
4     input wire [31:0] csr_wdata,
5     input wire      csr_we,
6     input wire [1:0] csr_op,      // 01=CSRRW, 10=CSRRS, 11=CSRRC
7     output wire [31:0] csr_rdata,
8     // 硬件端口（陷阱/SRET）
9     input wire      trap_enter_s,
10    input wire [31:0] trap_cause, trap_epc, trap_tval,
11    input wire      sret_exec,
12    output wire [1:0] priv_level

```

```

13 );
14 // mstatus 寄存器 (包含 sstatus 的位)
15 reg [31:0] mstatus;
16 reg [31:0] sepc, scause, stval, stvec, sscratch, satp;
17 reg [31:0] sie, sip;
18
19 // 读逻辑: 根据地址选择
20 reg [31:0] rdata;
21 always @(*) begin
22     case (csr_addr)
23         12'h100: rdata = {mstatus[8], mstatus[5], 20'b0,
24                         mstatus[1]}; // sstatus (简化)
25         12'h105: rdata = stvec;
26         12'h140: rdata = sscratch;
27         12'h141: rdata = sepc;
28         12'h142: rdata = scause;
29         12'h143: rdata = stval;
30         12'h180: rdata = satp;
31         12'h104: rdata = sie;
32         12'h144: rdata = sip;
33         default: rdata = 32'b0;
34     endcase
35 end
36 assign csr_rdata = rdata;
37
38 // 写逻辑
39 wire [31:0] write_val;
40 assign write_val = (csr_op == 2'b01) ? csr_wdata : // CSRRW
41                    (csr_op == 2'b10) ? (rdata | csr_wdata) : // CSRRS
42                    (csr_op == 2'b11) ? (rdata & ~csr_wdata) : // CSRRC
43                    rdata; // 读不改
44
45 always @(posedge clk) begin
46     if (trap_enter_s) begin
47         // 硬件写入陷阱上下文
48         sepc    <= trap_epc;
49         scause <= trap_cause;
50         stval  <= trap_tval;
51         mstatus[5] <= mstatus[1]; // SPIE <- SIE
52         mstatus[1] <= 1'b0;       // SIE  <- 0
53         mstatus[8] <= priv_level[0]; // SPP <- CPL
54     end else if (sret_exec) begin
55         // SRET 硬件操作
56         mstatus[1] <= mstatus[5]; // SIE  <- SPIE
57         mstatus[5] <= 1'b1;       // SPIE <- 1

```

```

58         mstatus[8] <= 1'b0;           // SPP <- 0 (U)
59     end else if (csr_we) begin
60         case (csr_addr)
61             12'h100: begin // sstatus -> mstatus
62                 mstatus[1] <= write_val[1]; // SIE
63                 mstatus[5] <= write_val[5]; // SPIE
64                 mstatus[8] <= write_val[8]; // SPP
65             end
66             12'h105: stvec <= write_val;
67             12'h140: sscratch <= write_val;
68             12'h141: sepc <= write_val;
69             12'h142: scause <= write_val;
70             12'h143: stval <= write_val;
71             12'h180: satp <= write_val;
72             12'h104: sie <= write_val;
73             12'h144: sip <= write_val;
74         endcase
75     end
76 end
77
78 assign priv_level = mstatus[8] ? 2'b01 : 2'b00; // SPP
79 endmodule

```

7 陷阱处理硬件

7.1 陷阱检测逻辑

陷阱检测发生在流水线的执行阶段。硬件需要检测以下事件：

事件	检测条件
ECALL	指令 opcode = 1110011, funct12 = 0x000
EBREAK	指令 opcode = 1110011, funct12 = 0x001
SRET	指令 opcode = 1110011, funct12 = 0x102
非法指令	解码失败或特权级不足
地址对齐错误	访存地址未对齐
缺页	地址翻译失败
中断	sip & sie & sstatus.SIE ≠ 0

7.2 陷阱状态机

当检测到陷阱时，硬件进入陷阱处理状态机：

1. **检测**：执行阶段检测到陷阱事件，确定 cause。
2. **委托判断**：读取 medeleg/mideleg，决定进入 S 还是 M 模式。

3. **保存上下文**: 写入 `sepc/scause/stval` (S 模式) 或 `mepc/mcause/mtval` (M 模式)。
4. **更新状态**: $SPIE \leftarrow SIE$; $SIE \leftarrow 0$; $SPP \leftarrow CPL$ 。
5. **跳转**: $PC \leftarrow stvec.BASE$ (Direct 模式) 或 $stvec.BASE + 4 * cause$ (Vectored 模式)。

7.3 SRET 执行硬件

SRET 的硬件操作:

1. 检查 `sstatus.TSR`: 如果为 1, 触发非法指令异常。
2. $CPL \leftarrow sstatus.SPP$ 。
3. $PC \leftarrow sepc$ 。
4. $sstatus.SIE \leftarrow sstatus.SPIE$ 。
5. $sstatus.SPIE \leftarrow 1$ 。
6. $sstatus.SPP \leftarrow 0$ 。

SRET 需要一个时钟周期完成状态更新, 然后从 `sepc` 取指。在流水线中, SRET 通常需要冲刷流水线 (因为 PC 跳转到 `sepc`)。

8 页表遍历器 (PTW) 设计

8.1 PTW 的整体架构

页表遍历器 (Page Table Walker, PTW) 是硬件中负责 Sv32 地址翻译的模块。当 TLB 未命中时, PTW 接管, 从内存中读取页表项, 完成两级遍历。

Listing 4: PTW 状态机 (伪代码)

```

1 // PTW 状态
2 parameter IDLE    = 0;
3 parameter LVL1    = 1; // 读取一级页表
4 parameter LVL2    = 2; // 读取二级页表
5 parameter DONE    = 3; // 翻译完成
6 parameter FAULT   = 4; // 缺页错误
7
8 // 输入: vaddr (虚拟地址), access_type (读/写/执行)
9 // 输出: paddr (物理地址), fault (是否缺页)
10
11 state IDLE:
12     if (tlb_miss) begin
13         // 计算一级 PTE 地址
14         pte_addr = satp.PPN << 12 | vaddr[31:22] << 2;
15         mem_req(pte_addr, READ);
16         state = LVL1;
17     end
18
19 state LVL1:

```

```

20     pte = mem_resp;
21     if (pte.V == 0) { fault = 1; state = FAULT; }
22     else if (pte.R || pte.W || pte.X) {
23         // 一级 PTE 是叶子节点 (大页 4MB)
24         // 检查权限...
25         paddr = {pte.PPN, vaddr[21:0]};
26         state = DONE;
27     } else {
28         // 非叶子节点, 读二级页表
29         pte_addr = pte.PPN << 12 | vaddr[21:12] << 2;
30         mem_req(pte_addr, READ);
31         state = LVL2;
32     }
33
34 state LVL2:
35     pte = mem_resp;
36     if (pte.V == 0) { fault = 1; state = FAULT; }
37     else {
38         // 检查权限...
39         paddr = {pte.PPN, vaddr[11:0]};
40         state = DONE;
41     }
42
43 state DONE:
44     // 将结果写入 TLB
45     tlb_write(vaddr[31:12], paddr[33:12], pte_perm);
46     state = IDLE;
47
48 state FAULT:
49     // 触发缺页异常
50     raise_page_fault(vaddr, access_type);
51     state = IDLE;

```

8.2 PTW 与流水线的交互

PTW 遍历页表需要多次内存访问 (Sv32 最多 2 次), 每次访问可能需要多个时钟周期。在遍历期间, 流水线需要暂停 (stall) 等待翻译完成。

两种集成策略:

策略一: 全流水线暂停。当 TLB 未命中时, 整个流水线暂停, PTW 独占总线进行遍历。完成后, 翻译结果写入 TLB, 流水线恢复。实现简单, 但暂停期间所有指令都停顿。

策略二: 仅暂停请求翻译的指令。其他指令继续执行 (如果它们的地址已在 TLB 中)。需要更复杂的流水线控制 (乱序完成、精确异常处理)。适合高性能处理器, 初学者不推荐。

8.3 缺页异常的生成

当 PTW 发现翻译失败时 (PTE 无效、权限不足、A/D 位未设置), 生成缺页异常:

- $stval \leftarrow$ 触发缺页的虚拟地址。
- $scause \leftarrow$ 12（指令获取）、13（Load）或 15（Store/AMO）。
- $sepc \leftarrow$ 触发缺页的指令 PC。
- 触发 S 模式陷阱（如果 `medeleg` 对应位为 1）。

9 TLB 设计

9.1 TLB 的基本结构

TLB（Translation Lookaside Buffer）是页表的硬件缓存，缓存最近使用的虚拟地址到物理地址的映射。没有 TLB，每次内存访问都需要 PTW 遍历（2 次内存读取），性能不可接受。

一个简单的直接映射 TLB 的结构：

字段	说明
valid	条目是否有效
vpn	虚拟页号（20 位）
asid	地址空间标识符（9 位）
ppn	物理页号（22 位）
perm	权限位（R/W/X/U/G/A/D）

9.2 TLB 查询与命中

TLB 查询逻辑：

Listing 5: TLB 查询逻辑

```

1 function tlb_lookup(vaddr, current_asid);
2     vpn = vaddr[31:12];
3     for (i = 0; i < TLB_SIZE; i++) begin
4         if (tlb[i].valid &&
5             tlb[i].vpn == vpn &&
6             (tlb[i].global || tlb[i].asid == current_asid)) begin
7             // 命中
8             return {tlb[i].ppn, tlb[i].perm, HIT};
9         end
10    end
11    return MISS;
12 endfunction

```

9.3 SFENCE.VMA 的 TLB 刷新

SFENCE.VMA 指令根据 `rs1` 和 `rs2` 的值执行不同范围的 TLB 刷新：

在 RTL 中，全量刷新最简单（所有 `valid` 位清零），精确刷新需要遍历 TLB 比较条目。

rs1	rs2 刷新范围	
x0	x0	所有 ASID 的所有条目（全量刷新）
≠x0	x0	所有 ASID 中 VPN=rs1 的条目
x0	≠x0	ASID=rs2 的所有条目
≠x0	≠x0	ASID=rs2 中 VPN=rs1 的条目

9.4 ASID 支持

ASID（地址空间标识符）让 TLB 在进程切换时不需要全量刷新。每个 TLB 条目带有一个 ASID 标签，查询时只匹配当前 `satp.ASID` 的条目（全局页 `G=1` 除外）。

ASID 的工作流程：进程 A（ASID=1）运行时，TLB 填充 ASID=1 的条目。切换到进程 B（ASID=2）时，`satp.ASID` 改为 2，TLB 查询时只匹配 ASID=2 的条目——ASID=1 的条目虽然还在 TLB 中，但不会被命中。切回进程 A 时，ASID=1 的条目仍然有效，不需要重新遍历页表。

ASID 耗尽问题：ASID 只有 9 位（512 个值），如果系统中有超过 512 个进程，ASID 会复用。当 ASID 复用时，OS 需要执行 `SFENCE.VMA` 刷新该 ASID 的所有条目，避免新旧进程的翻译混淆。这叫做 ASID 回收（ASID recycling），是 OS 内存管理的职责。

10 地址翻译与流水线集成

10.1 取指阶段的翻译

每条指令的取指都需要将 PC（虚拟地址）翻译为物理地址。I-TLB（Instruction TLB）缓存指令地址的翻译结果。如果 I-TLB 命中，取指延迟增加 0 个周期（并行查询）；如果未命中，需要 PTW 遍历，延迟增加数十个周期。

10.2 访存阶段的翻译

Load/Store 指令的有效地址（虚拟地址）需要通过 D-TLB（Data TLB）翻译。翻译流程：

1. 计算 `vaddr = base_reg + offset`。
2. 查询 D-TLB： `paddr = {TLB.PPN, vaddr[11:0]}`；检查 R/W 权限。
3. TLB 未命中：触发 PTW，等待翻译完成。
4. 权限不足：Load 触发 `cause=13`，Store 触发 `cause=15`。

10.3 翻译与 PMP 的顺序

当 `satp.MODE=Sv32` 时，S/U 模式的访存先经过 Sv32 翻译（虚拟地址 → 物理地址），再经过 PMP 检查。M 模式的访存不经过 Sv32 翻译，直接经过 PMP 检查。

翻译与 PMP 的顺序很重要：先翻译得到物理地址，再用物理地址查 PMP。这意味着 PMP 检查的是物理地址的权限，与虚拟地址无关。OS 可以通过 PMP 限制 S 模式访问某些物理地址区域（如 M 模式固件代码区），即使 S 模式的页表映射了这些区域。

10.4 性能考量与优化

优化一：增大 TLB 容量。更多条目意味着更高命中率，但面积和功耗也更大。典型 D-TLB 64 条目，I-TLB 32 条目。

优化二：组相联 TLB。直接映射 TLB 容易冲突，组相联（2 路或 4 路）可以减少冲突。

优化三：大页支持。Sv32 支持大页（4 MiB，一级 PTE 作为叶子节点）。大页用一个 TLB 条目覆盖 4 MiB 空间，减少 TLB 条目消耗。

优化四：ASID 支持。避免进程切换时的全量 TLB 刷新，保留旧进程的翻译。

优化五：预取翻译。当检测到顺序访问模式时，提前翻译下一页，避免 TLB 未命中的停顿。

对初学者，先实现一个简单的直接映射 TLB（16 条目，无 ASID），跑通基本功能。性能优化可以在功能正确后再做。记住一个原则：正确性优先，性能其次。

Part III

操作系统软件编程

本部分从操作系统开发者的视角，讲解如何编程使用第一部分定义的 S 模式 ISA。我们将覆盖从 M 模式到 S 模式的启动流程、OS 的陷阱处理框架、页表管理，并给出完整的汇编/C 代码示例。这些代码可以直接用于你自己的教学 OS，或作为理解 Linux/xv6 内核启动流程的参考。

11 从 M 模式到 S 模式的启动

11.1 启动的整体流程

RISC-V 系统的启动是一个从 M 模式到 S 模式再到 U 模式的逐级下降过程。处理器复位后从 M 模式开始执行（PC = RESET_VECTOR，通常是 0x80000000 或 0x1000），M 模式固件（如 OpenSBI）完成硬件初始化后，配置好委托和 PMP，然后跳转到 S 模式的 OS 入口。OS 在 S 模式完成自己的初始化（设置 stvec、建立页表、启用中断），然后创建第一个用户进程，切换到 U 模式。

典型启动链：ROM Bootloader → OpenSBI（M 模式）→ Linux/xv6 内核（S 模式）→ 用户程序（U 模式）。每一级负责初始化自己管理的资源，然后下降到下一级。

11.2 OpenSBI 的角色

OpenSBI 是 RISC-V 官方的 M 模式固件实现，运行在 M 模式，为 S 模式 OS 提供 SBI（Supervisor Binary Interface）服务。SBI 是一个标准化的系统调用接口——OS 通过 ECALL 请求 OpenSBI 执行特权操作（如设置定时器、发送 IPI、读写设备）。

OpenSBI 在启动时完成以下工作：

1. 硬件初始化：配置 PMP 允许 S 模式访问全部物理内存。
2. 委托配置：设置 medeleg/mideleg，将 S 级中断和缺页异常委托给 S 模式。

3. 定时器初始化：设置 `mtimecmp`，启用 M 级定时器中断。
4. 跳转到 S 模式：设置 `mepc` 为 OS 入口地址，执行 `MRET`。

11.3 M 模式到 S 模式的跳转代码

以下是一个最小化的 $M \rightarrow S$ 跳转代码（类似 OpenSBI 的核心逻辑）：

Listing 6: M 模式到 S 模式的跳转

```

1  # 入口：M 模式
2  m_start:
3      # 1. 配置 PMP：允许 S 模式访问全部地址
4      csrw pmpaddr0, 0x3fffffff # 34 位全 1 (最大范围)
5      csrw pmpcfg0, 0x0f # R=1 W=1 X=1, L=0, MODE=NAPOT
6
7      # 2. 配置异常/中断委托
8      la t0, medeleg_bits
9      ld t0, 0(t0)
10     csrw medeleg, t0 # 委托异常给 S 模式
11     la t0, mideleg_bits
12     ld t0, 0(t0)
13     csrw mideleg, t0 # 委托中断给 S 模式
14
15     # 3. 设置 S 模式入口地址
16     la t0, s_mode_entry # OS 内核入口
17     csrw mepc, t0
18
19     # 4. 设置 mstatus.MPP = 1 (S 模式)
20     li t0, 0x800 # MPP = 01 (S mode)
21     csrs mstatus, t0 # 设置 MPP
22
23     # 5. 执行 MRET，跳转到 S 模式
24     mret
25
26 # 数据
27 .section .data
28 medeleg_bits: .quad 0xB1FF # 委托 cause 0,2,3,5,7,8,12-15
29 mideleg_bits: .quad 0x0222 # 委托 SSI(1), STI(5), SEI(9)

```

11.4 S 模式入口的初始化

OS 内核在 S 模式入口处需要完成：

Listing 7: S 模式入口初始化

```

1 s_mode_entry:
2     # 1. 设置 S 模式栈
3     la sp, kernel_stack_top

```

```

4
5     # 2. 设置 stvec (陷阱向量)
6     la t0, trap_vector
7     csrw stvec, t0
8
9     # 3. 清除 SIE (暂时禁用中断)
10    csrw sie, zero
11
12    # 4. 初始化页表 (c 函数)
13    call paging_init
14
15    # 5. 启用 S 级中断
16    li t0, 0x222                # SSIE | STIE | SEIE
17    csrs sie, t0
18    csrs sstatus, 2            # SIE = 1
19
20    # 6. 创建第一个用户进程
21    call user_init
22
23    # 7. 首次调度 (SRET 到 U 模式)
24    call schedule

```

12 OS 陷阱处理框架

12.1 stvec 的设置与陷阱入口

OS 在启动时设置 stvec 指向陷阱入口。陷阱入口是一段汇编代码，负责保存上下文、调用 C 处理函数、恢复上下文。

12.2 寄存器保存与恢复

陷阱入口的核心是保存所有通用寄存器到内核栈 (trapframe)，处理完毕后恢复。使用 sscratch 保存内核栈指针：

Listing 8: 陷阱入口与出口 (汇编)

```

1 .globl trap_vector
2 trap_vector:
3     # 1. 交换 sp 和 sscratch, 获取内核栈
4     csrrw sp, sscratch, sp
5
6     # 2. 保存通用寄存器到 trapframe
7     addi sp, sp, -256
8     sd ra, 0(sp)
9     sd gp, 8(sp)
10    sd tp, 16(sp)
11    sd t0, 24(sp)

```

```

12  sd t1, 32(sp)
13  # ... 保存 t2-t6, a0-a7, s0-s11 ...
14  sd s11, 248(sp)
15
16  # 3. 保存 sepc 和 sstatus
17  csrr t0, sepc
18  sd t0, 256(sp)          # 保存 sepc 到 trapframe
19  csrr t1, sstatus
20  sd t1, 264(sp)          # 保存 sstatus
21
22  # 4. 调用 C 陷阱处理函数
23  mv a0, sp                # trapframe 指针作为参数
24  call trap_handler
25
26  # 5. 恢复 sepc 和 sstatus
27  ld t0, 256(sp)
28  csrwr sepc, t0
29  ld t1, 264(sp)
30  csrwr sstatus, t1
31
32  # 6. 恢复通用寄存器
33  ld ra, 0(sp)
34  ld gp, 8(sp)
35  # ... 恢复所有寄存器 ...
36  ld s11, 248(sp)
37  addi sp, sp, 256
38
39  # 7. 交换回 sp 和 sscratch
40  csrrw sp, sscratch, sp
41
42  # 8. 返回 (SRET)
43  sret

```

12.3 scause 分发逻辑

C 陷阱处理函数读取 scause，判断是中断还是异常，然后分发：

Listing 9: 陷阱处理函数 (C)

```

1  #include <stdint.h>
2
3  struct trapframe {
4      uint64_t ra, gp, tp;
5      uint64_t t0, t1, t2, t3, t4, t5, t6;
6      uint64_t a0, a1, a2, a3, a4, a5, a6, a7;
7      uint64_t s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;
8      uint64_t sepc;

```



```

9     uint64_t sstatus;
10 };
11
12 void trap_handler(struct trapframe *tf) {
13     uint64_t cause = read_csr(scause);
14
15     if (cause & (1ULL << 63)) {
16         // 中断 (最高位 = 1)
17         switch (cause & 0x7FFFFFFF) {
18             case 1: // S 级软件中断
19                 handle_ssi(tf);
20                 break;
21             case 5: // S 级定时器中断
22                 handle_sti(tf);
23                 break;
24             case 9: // S 级外部中断
25                 handle_sei(tf);
26                 break;
27             default:
28                 panic("未知中断");
29         }
30     } else {
31         // 异常 (最高位 = 0)
32         switch (cause) {
33             case 8: // U 模式 ECALL (系统调用)
34                 tf->sepc += 4; // 跳过 ECALL 指令
35                 handle_syscall(tf);
36                 break;
37             case 12: // 指令缺页
38             case 13: // Load 缺页
39             case 15: // Store 缺页
40                 handle_page_fault(tf);
41                 break;
42             case 2: // 非法指令
43                 handle_illegal(tf);
44                 break;
45             default:
46                 panic("未知异常");
47         }
48     }
49 }

```

12.4 系统调用处理

Listing 10: 系统调用处理 (C)

```

1 // 系统调用号
2 #define SYS_write    1
3 #define SYS_read     2
4 #define SYS_exit     3
5 #define SYS_getpid   4
6 #define SYS_fork     5
7 #define SYS_exec     6
8
9 void handle_syscall(struct trapframe *tf) {
10     int syscall_num = tf->a7;    // a7 = 系统调用号
11     uint64_t ret;
12
13     switch (syscall_num) {
14         case SYS_write:
15             ret = sys_write(tf->a0, (char*)tf->a1, tf->a2);
16             break;
17         case SYS_read:
18             ret = sys_read(tf->a0, (char*)tf->a1, tf->a2);
19             break;
20         case SYS_exit:
21             sys_exit(tf->a0);
22             break; // 不会返回
23         case SYS_getpid:
24             ret = sys_getpid();
25             break;
26         case SYS_fork:
27             ret = sys_fork();
28             break;
29         case SYS_exec:
30             ret = sys_exec((char*)tf->a0);
31             break;
32         default:
33             ret = -1; // 未知系统调用
34     }
35     tf->a0 = ret; // 返回值放入 a0
36 }

```

12.5 中断处理

Listing 11: 中断处理 (C)

```

1 void handle_sti(struct trapframe *tf) {
2     // S 级定时器中断
3     // 1. 读取当前时间
4     uint64_t now = read_time();

```

```

5 // 2. 设置下一次定时器 (10ms 后)
6 set_timer(now + TIMER_INTERVAL);
7 // 3. 调度器: 可能切换进程
8 schedule();
9 }
10
11 void handle_ssi(struct trapframe *tf) {
12     // S 级软件中断 (IPI)
13     // 1. 清除 SSIP
14     write_csr(sip, read_csr(sip) & ~2);
15     // 2. 处理 IPI 消息
16     ipi_handle();
17 }
18
19 void handle_sei(struct trapframe *tf) {
20     // S 级外部中断
21     // 1. 从 PLIC claim 中断
22     int irq = plic_claim();
23     // 2. 分发到设备驱动
24     if (irq == UART_IRQ) {
25         uart_interrupt();
26     } else if (irq == DISK_IRQ) {
27         disk_interrupt();
28     }
29     // 3. 通知 PLIC 处理完成
30     plic_complete(irq);
31 }

```

13 OS 页表管理

13.1 页表的分配与初始化

OS 在创建进程时分配根页表:

Listing 12: 页表分配与初始化 (C)

```

1 // 分配一个 4KB 页表 (1024 个 PTE)
2 uint32_t *alloc_page_table() {
3     uint32_t *pt = (uint32_t*)alloc_physical_page();
4     // 初始化所有 PTE 为 0 (V=0)
5     for (int i = 0; i < 1024; i++) {
6         pt[i] = 0;
7     }
8     return pt;
9 }
10

```

```

11 // 创建新进程的地址空间
12 pagetable_t proc_create_pagetable() {
13     return alloc_page_table(); // 返回根页表
14 }

```

13.2 PTE 的填充

Listing 13: PTE 填充 (C)

```

1 // PTE 位定义
2 #define PTE_V (1 << 0) // 有效
3 #define PTE_R (1 << 1) // 可读
4 #define PTE_W (1 << 2) // 可写
5 #define PTE_X (1 << 3) // 可执行
6 #define PTE_U (1 << 4) // 用户可访问
7 #define PTE_A (1 << 6) // 已访问
8 #define PTE_D (1 << 7) // 脏位
9
10 // 构造叶子 PTE
11 uint32_t make_leaf_pte(uint32_t ppn, int perm) {
12     return (ppn << 10) | perm | PTE_V | PTE_A | PTE_D;
13 }
14
15 // 构造非叶子 PTE (指向下一级页表)
16 uint32_t make_intermediate_pte(uint32_t child_pt_ppn) {
17     return (child_pt_ppn << 10) | PTE_V; // V=1, R=W=X=0
18 }

```

13.3 虚拟地址映射

Listing 14: 虚拟地址映射 (C)

```

1 // 将虚拟地址 va 映射到物理地址 pa, 权限为 perm
2 int map_page(pagetable_t pt, uint32_t va, uint32_t pa, int perm) {
3     // 第一级: VPN[1] = va[31:22]
4     uint32_t *pte1 = &pt[va >> 22];
5     uint32_t *level2_pt;
6
7     if (!(*pte1 & PTE_V)) {
8         // 一级 PTE 无效, 分配二级页表
9         level2_pt = alloc_page_table();
10        *pte1 = make_intermediate_pte((uint32_t)level2_pt >> 12);
11    } else {
12        // 一级 PTE 有效, 获取二级页表地址
13        level2_pt = (uint32_t*)(((*pte1) >> 10) << 12);
14    }

```

```

15
16 // 第二级: VPN[0] = va[21:12]
17 uint32_t *pte2 = &level2_pt[(va >> 12) & 0x3FF];
18 if (*pte2 & PTE_V) {
19     return -1; // 已映射, 错误
20 }
21
22 // 填充二级 PTE (叶子节点)
23 *pte2 = make_leaf_pte(pa >> 12, perm);
24 return 0;
25 }

```

13.4 satp 切换与 SFENCE.VMA

Listing 15: 进程切换的 satp 更新

```

1 # 切换到新进程的页表
2 # a0 = 新进程的根页表物理地址
3 # a1 = 新进程的 ASID
4 switch_pagetable:
5     # 构造 satp 值: MODE=1(Sv32) | ASID | PPN
6     slli a1, a1, 22      # ASID 移到 bit 30:22
7     srli a0, a0, 12      # 物理地址 -> PPN
8     ori  a0, a0, (1 << 31) # 设置 MODE=1 (Sv32)
9     or   a0, a0, a1      # 合并 ASID
10    csrw satp, a0        # 写入 satp
11    sfence.vma zero, zero # 刷新 TLB (无 ASID 时全量刷新)
12    ret

```

13.5 缺页处理

Listing 16: 缺页处理 (C)

```

1 void handle_page_fault(struct trapframe *tf) {
2     uint64_t cause = read_csr(scause);
3     uint64_t fault_addr = read_csr(stval); // 缺页地址
4
5     // 判断缺页类型
6     int is_write = (cause == 15); // Store 缺页
7     int is_read  = (cause == 13); // Load 缺页
8     int is_exec  = (cause == 12); // 指令缺页
9
10    // 查找虚拟内存区域 (VMA)
11    struct vma *vma = find_vma(current->mm, fault_addr);
12    if (vma == NULL) {
13        // 地址不在任何 VMA 中, 段错误

```

```

14     kill_process(current, SIGSEGV);
15     return;
16 }
17
18 // 分配物理页
19 uint32_t pa = alloc_physical_page();
20
21 // 如果是文件映射页，从文件读取数据
22 if (vma->file) {
23     read_from_file(vma->file, pa, vma->offset, PAGE_SIZE);
24 }
25
26 // 映射虚拟地址到物理页
27 int perm = PTE_V | PTE_U | PTE_A;
28 if (vma->flags & VM_WRITE) perm |= PTE_W | PTE_D;
29 if (vma->flags & VM_READ)  perm |= PTE_R;
30 if (vma->flags & VM_EXEC)  perm |= PTE_X;
31
32 map_page(current->pagetable, fault_addr & ~0xFFF, pa, perm);
33
34 // 刷新 TLB
35 asm volatile("sfence.vma %0, zero" :: "r"(fault_addr));
36 }

```

14 完整代码示例

14.1 最小陷阱处理程序

以下是一个完整的最小陷阱处理程序，包含陷阱入口、上下文保存、C 分发、系统调用处理：

Listing 17: 最小陷阱处理程序（汇编入口）

```

1 .section .text
2 .globl trap_vector
3 trap_vector:
4     csrrw sp, sscratch, sp      # 获取内核栈
5     addi  sp, sp, -256
6
7     # 保存所有通用寄存器
8     sd ra,  0(sp)
9     sd t0,  8(sp)
10    sd t1, 16(sp)
11    sd t2, 24(sp)
12    sd t3, 32(sp)
13    sd t4, 40(sp)

```

```

14     sd t5, 48(sp)
15     sd t6, 56(sp)
16     sd a0, 64(sp)
17     sd a1, 72(sp)
18     sd a2, 80(sp)
19     sd a3, 88(sp)
20     sd a4, 96(sp)
21     sd a5, 104(sp)
22     sd a6, 112(sp)
23     sd a7, 120(sp)
24     # s0-s11 省略...
25
26     # 保存 sepc
27     csrr t0, sepc
28     sd t0, 248(sp)
29
30     # 调用 C 处理函数
31     mv a0, sp
32     call trap_handler
33
34     # 恢复 sepc
35     ld t0, 248(sp)
36     csrw sepc, t0
37
38     # 恢复寄存器
39     ld ra, 0(sp)
40     ld t0, 8(sp)
41     # ... 恢复所有 ...
42     addi sp, sp, 256
43
44     # 交换回 sp
45     csrrw sp, sscratch, sp
46     sret

```

14.2 页表初始化示例

Listing 18: 内核页表初始化 (C)

```

1 void paging_init(void) {
2     pagetable_t pt = alloc_page_table();
3
4     // 1. 映射内核代码段 (S 模式, 可读可执行)
5     for (uint32_t va = KERNEL_START; va < KERNEL_END; va += PAGE_SIZE) {
6         map_page(pt, va, va, PTE_R | PTE_X); // 恒等映射
7     }
8

```

```

9 // 2. 映射内核数据段 (S 模式, 可读可写)
10 for (uint32_t va = DATA_START; va < DATA_END; va += PAGE_SIZE) {
11     map_page(pt, va, va, PTE_R | PTE_W);
12 }
13
14 // 3. 映射 UART (S 模式, 可读可写)
15 map_page(pt, UART_BASE, UART_BASE, PTE_R | PTE_W);
16
17 // 4. 激活页表
18 uint32_t satp_val = (1 << 31) | ((uint32_t)pt >> 12);
19 asm volatile("csrw satp, %0" :: "r"(satp_val));
20 asm volatile("sfence.vma zero, zero");
21 }

```

14.3 系统调用处理示例

Listing 19: 系统调用处理 (C)

```

1 #define SYS_write 1
2
3 int64_t sys_write(int fd, const char *buf, uint64_t count) {
4     if (fd != 1 && fd != 2) return -1; // 只支持 stdout/stderr
5
6     for (uint64_t i = 0; i < count; i++) {
7         uart_putc(buf[i]); // 逐字符输出到 UART
8     }
9     return count;
10 }
11
12 void handle_syscall(struct trapframe *tf) {
13     tf->sepc += 4; // 跳过 ECALL
14
15     switch (tf->a7) {
16         case SYS_write:
17             tf->a0 = sys_write(tf->a0, (char*)tf->a1, tf->a2);
18             break;
19         default:
20             tf->a0 = -1;
21     }
22 }

```

14.4 端到端流程：一次系统调用的完整路径

当用户程序调用 `write(1, "hello", 5)` 时：

1. 用户程序 (U 模式) 将参数装入 `a0=1, a1="hello", a2=5, a7=1 (SYS_write)`，执行 ECALL。

2. 硬件检测到 ECALL, 生成 `cause=8` 的异常。检查 `medeleg[8]=1` (已委托), 决定进入 S 模式。
3. 硬件保存上下文: $\text{sepc} \leftarrow \text{ECALL 的 PC}$, $\text{scause} \leftarrow 8$, $\text{sstatus.SPP} \leftarrow 0$ (U), $\text{sstatus.SIE} \leftarrow 0$ 。 $\text{PC} \leftarrow \text{stvec}$ 。
4. 陷阱入口 (`trap_vector`) 执行: 交换 `sp/sscratch` 获取内核栈, 保存通用寄存器到 `trapframe`。
5. 调用 `trap_handler` (C 函数): 读 `scause=8`, 判断是系统调用, 调用 `handle_syscall`。
6. `handle_syscall`: $\text{sepc} += 4$ (跳过 ECALL), 读 `a7=1` (`SYS_write`), 调用 `sys_write(1, "hello", 5)`。
7. `sys_write` 向 UART 写字符串, 返回 5 (写入字节数)。返回值放入 `tf→a0`。
8. `trap_handler` 返回, `trap_vector` 恢复寄存器 (包括 `a0=5`), 写回 `sepc` 和 `sstatus`。
9. 执行 SRET: 硬件恢复 $\text{PC} \leftarrow \text{sepc}$ (ECALL 的下一条指令), $\text{CPL} \leftarrow \text{SPP}=0$ (U 模式), $\text{sstatus.SIE} \leftarrow \text{SPIE}$ 。
10. 用户程序继续执行, `a0` 中是系统调用的返回值 5。

这个流程展示了 RISC-V S 模式架构的精髓: 硬件提供机制 (CSR、陷阱、委托、特权指令), 软件提供策略 (系统调用分发、权限检查、资源管理)。硬件和软件各司其职, 共同构建了一个安全、高效的操作系统运行环境。理解了这个流程, 你就理解了 S 模式架构的核心——这也是「一生一芯」计划希望学习者达到的认知水平。